Design and Implementation of a Non-Relational Database in MongoDB

Panagiotis Athanasopoulos
University of Piraeus, Department of Digital Systems
Piraeus, Greece
apanagiotis170@gmail.com

Marina Nouchou
University of Piraeus, Department of Digital Systems
Piraeus, Greece
marinanuhu4598@gmail.com

Anastasia Achilleos
University of Piraeus, Department of Digital Systems
Piraeus, Greece
anactaciaachilleos@gmail.com

Abstract

We present the design and implementation of a non-relational MongoDB database for AIS messages in the Saronic Gulf (May 2017–December 2019). We modeled the data using five main collections (`vessels`, `ais_dynamic`, `ais_synopses`, `ports/harbours`, and `regions`) and targeted indexes, including `2dsphere`, in order to support: (a) relational-style queries, (b) spatial queries (*within a circle*, *K nearest*), and (c) spatio-temporal queries (proximity X within time interval T). We evaluate the impact of indexing and sharding choices on response time and demonstrate substantial latency improvements in spatial and temporal scenarios. The dataset is available through Zenodo [4].

1 Introduction

Big Data analysis is a challenge faced by an increasing number of scientific disciplines. The term Big Data refers to datasets that are difficult to manage using traditional analysis methods and are commonly characterized by three main properties: large volume, high variety, and high velocity of generation. One such problem, investigated in this work, is the management of large-scale maritime navigation data. Maritime transport plays a major role in the global economy and, beyond tourism, is responsible for approximately 80% of world trade. Furthermore, the effective management of maritime traffic is particularly important because navigation is not an isolated activity; rather, it is part of a broader system influenced by several factors and must always be managed efficiently and safely [1] [2].

Among the various surveillance technologies used to identify and track vessels at sea in real time, the Automatic Identification System (AIS) has become a mandatory standard for most vessel categories, including cargo, passenger, and fishing vessels. AIS is based on the Global Positioning System (GPS), together with onboard sensors such as speed logs and compasses, and is currently used by more than half a million vessels worldwide to transmit their position [3].

In this work, the dataset under study contains AIS (Automatic Identification System) messages recorded in the wider Saronic Gulf area in Greece for approximately 2.5 years, from May 2017 to December 2019 [4]. The data were collected through an AIS station located at the University of Piraeus, approximately 1 km from the Port of Piraeus, and constitute a representative dataset for maritime activity analysis. The dataset contains three main categories of information:

(1) Navigation data, such as vessel positions, speeds, and courses.
(2) Geographic data, such as harbour locations and the geographical structure of the study area.
(3) Meteorological data associated with weather conditions during the observation period.

In addition, the dataset contains important metadata, including trajectory synopses that record critical points along vessel routes, such as changes in speed or course. These synopses are particularly useful for extracting higher-level information and supporting decision making.

The dataset contains more than 244 million AIS records, with an average of more than 10,000 records per hour. This makes it one of the largest and densest openly available AIS datasets for such a long observation period. Therefore, it constitutes an appropriate basis for large-scale mobility data processing and analysis [3].

1.2 Problem Definition

The problem addressed in this work concerns the management, processing, and analysis of a large volume of data originating from the Automatic Identification System (AIS) and describing maritime activity in the Saronic Gulf. The dataset is available at https://zenodo.org/records/6323416 [4]. The data include vessel positions, trajectory synopses, static vessel information and vessel types, geographic information such as islands and harbours, and meteorological data.

The main challenge is the management of a complex dataset containing heterogeneous and structured representations through a non-relational approach, specifically MongoDB. This requires appropriate data modeling, storage, indexing, and data partitioning across nodes so that the required queries can be executed efficiently. After modeling and uploading the data to MongoDB, the following query categories are investigated:

(1) Relational-style queries, such as finding vessels registered in Greece whose name contains a given alphanumeric term.
(2) Spatial queries, such as finding vessels inside a circle defined by a given center and radius, or finding the K nearest vessels to a given point.
(3) Spatio-temporal queries, such as finding vessels that approached each other within a distance X during a time interval T.

2 Data & Preprocessing

The schema of `zenodoDB` includes:

- **ais_dynamic**: dynamic positions/measurements (GeoJSON Point, timestamp, speed, course, heading, vessel_id).
- **ais_synopses**: summarized “events”/trajectory points per vessel (t, lon/lat or GeoJSON).

- **vessels**: static vessel information (vessel_id, name, country, MMSI/IMO, etc.).
- **ship_types**: vessel-type dictionary (code/group/label).
- **Geospatial collections**: `ports`, `harbours` (typically Point), `piraeus_port` (Polygon describing the port zone), `regions`, `islands`, `areas`, `spatial_coverage`, and `territorial_waters` (Polygon/MultiPolygon).

Preprocessing. (i) Coordinates were unified into GeoJSON [lon,lat], (ii) `timestamp` values were represented as `ISODate`, and (iii) the descriptive `ship_type` information could optionally be copied into `vessels` to enable fast filtering without `$lookup`.

Table 1: Summary characteristics of the collections after preprocessing

Collection	#Docs	Size	Description
vessels	«6230»	«GB»	Static vessel information, embedded `ship_type`
ais_dynamic	«4305035»	«GB»	AIS positions (GeoJSON Point), time-series
ais_synopses	«629523»	«GB»	Summarized events/trajectory points
ship_types	«6228»	«MB»	Vessel-type dictionary (code, group, label)
noaa_weather	«11160»	«MB»	Temporal/meteorological data (GeoJSON Point/Polygon)
harbours	«21»	«MB»	Harbour/mooring locations (GeoJSON Point)
piraeus_port	«1»	«MB»	Piraeus port zone (GeoJSON Polygon)
regions	«1»	«MB/GB»	Areas of interest (GeoJSON (Multi)Polygon)
islands	«136»	«MB»	Islands (GeoJSON Polygon)
spatial_coverage	«76»	«MB»	Receiver coverage (Polygon/MultiPolygon)
territorial_waters	«1»	«MB»	Territorial waters (Polygon/MultiPolygon)

3 Schema Design (Modeling)

The design of the collections is based on the principles of flexibility, scalability, and efficient storage of spatio-temporal data. Within this context, the following main design choices were adopted:

- **Embedding:** Fields such as `shiptype` in `vessels` and `vessel` in `ais_dynamic` are embedded in the documents, allowing fast filtering without requiring `$lookup`. The `vessel_id` remains the authoritative key.
- **Normalization.**
  For reusable or potentially changing values, such as vessel types (`ship_types`), separate collections are used to improve management and consistency.
- **Geospatial support:** All geometric fields follow the GeoJSON standard. Collections whose coordinates use WGS84, such as `ais_dynamic`, `harbours`, and `islands`, support `2dsphere` indexes. In cases such as `regions`, where projected coordinates are used, storing a WGS84 version of the geometry in a separate field, e.g. `geometry_wgs84`, is recommended for spatial querying.
- **Temporal data modeling:** The `timestamp` fields use ISODate for full compatibility with MongoDB, while in special cases such as `ais_synopses`, epoch time in milliseconds is used for compact representation and efficient calculations.
- **Use of FeatureCollections:** The `piraeus_port`, `regions`, and `spatial_coverage` collections adopt the GeoJSON `FeatureCollection` format to facilitate integration with GIS systems and compatibility with APIs such as Mapbox or Leaflet.
- **High-performance queries:** The design favors spatio-temporal queries (`$geoWithin`, `$near`, and range queries on `timestamp`) while keeping dependencies between collections limited, thereby reducing the need for `$lookup`.

This approach enables flexible and scalable management of maritime traffic data while maintaining high performance and facilitating future enrichment. The following subsections describe the main collections and their corresponding design choices in detail.

3.1 Collection `vessels`

The `vessels` collection stores static vessel characteristics such as flag state, vessel name, and identification codes. For efficiency, the `shiptype` field is embedded as a subdocument so that additional lookup operations can be avoided when queries are performed by vessel type.

Listing 1: Example document from the vessels collection (from the cluster)

```
{
  "vessel_id": "2c46b2c29086c03cb774fa3fbd09f4c22a793eb7e7403
       21b5f17518a343dfc1e",
  "country": "Albania",
  "shiptype": {
    "Type Code": 52,
    "Description": "Tug"
  }
}
```

3.2 Collection `ais_dynamic`

The `ais_dynamic` collection contains dynamic observations originating from AIS, namely the geographical position of each vessel in GeoJSON Point format, course, speed, heading, and timestamp. For flexibility and performance, each document can also include an optional embedded snapshot containing basic vessel information, allowing fast filters without the use of a lookup. Nevertheless, `vessel_id` remains the main reference key and is used to associate the record with the remaining vessel data.

Listing 2: Example document from the ais_dynamic collection (MongoDB Extended JSON)

```
{
  "_id": { "$oid": "676fecab1ce3cbb7e4ac14e9" },
  "course": 359.6,
  "geometry": {
    "type": "Point",
    "coordinates": [ 23.6350566666667, 37.943545 ]
```

```
  },
  "heading": 219,
  "speed": 0,
  "timestamp": { "$date": "2017-05-09T14:05:26.000Z" },
  "vessel": {
    "vessel_id": "0fe51bee65c87f2f06f33d5b42fd47dfaf7d99c7233
        6e2ec5543f709f90a2d9b",
    "country": "Greece",
    "shipType": {}
  },
  "vessel_id": "0fe51bee65c87f2f06f33d5b42fd47dfaf7d99c72336e
      2ec5543f709f90a2d9b"
}
```

3.3 Collection `ais_synopses`

The `ais_synopses` collection stores summarized events extracted from vessel trajectory analysis in order to provide a more compact and targeted representation than the raw observations stored in `ais_dynamic`. Such events may include large reception gaps, sudden changes in speed or course, stops, or approaches to areas of interest.

Listing 3: Example document from the ais_synopses collection (MongoDB Extended JSON from the cluster)

```
{
  "_id": { "$oid": "6782303890f33466a5c174ad" },
  "t": { "$numberLong": "1501534800000" },
  "vessel_id": "9e854ccb6855e244bd694010fcb8300633e1b1b631723
      a50f04765521c8f3a10",
  "lon": 23.6829416667,
  "lat": 37.9291066667,
  "annotations": "['GAP_END']",
  "transport_trail": "[{'topic': 'datacsv_saronikos_3', '
      timestamp': 1501534800}]"
}
```

Comment: The collection stores trajectory "events"/points per vessel. The time field `t` is represented as `$numberLong` in milliseconds since the epoch. The `annotations` and `transport_trail` fields are *strings* containing list representations.

3.4 Collection `ship_types`

The `ship_types` collection acts as a reference dictionary for vessel types. Each document contains a unique type code and the corresponding description, e.g. cargo, tanker, or passenger. Normalizing this information into a separate collection ensures consistency and facilitates reuse by other collections such as `vessels`.

Listing 4: Example document from the ship_types collection (MongoDB Extended JSON)

```
{
  "_id": { "$oid": "67687ec46762f23d2f794183" },
  "Type Code": 21,
  "Description": "Wing in ground (WIG), Hazardous category A"
}
```

3.5 Collection `harbours`

The `harbours` collection contains geospatial data for harbours and marinas. Each document is stored as a GeoJSON Point corresponding to the geographical location of the harbour. Associated fields contain basic descriptive attributes, such as the harbour name and, when available, identification codes such as LOCODE.

The collection supports queries related to vessel entry into or exit from specific harbours, as well as traffic analysis in areas of intense maritime activity. By using `2dsphere` indexes, spatial queries such as detecting vessels within a short distance from a harbour can be executed efficiently.

Listing 5: Example document from the harbours collection (from the cluster)

```
{
  "_id": { "$oid": "67a12ad54c8acf03ab957eaa" },
  "geometry": {
    "type": "Point",
    "coordinates": [23.64899, 37.93486]
  },
  "properties": {
    "Lat": 37.93486,
    "Lon": 23.64899,
    "Port Name": "ZEA",
    "type": "Marine",
    "locode": "GRMAZ"
  }
}
```

3.6 Collection `piraeus_port`

The `piraeus_port` collection stores the geometric data that delineate the Port of Piraeus. Documents are represented as GeoJSON FeatureCollections with Polygon or MultiPolygon geometries so that the spatial extent of the port zone can be represented accurately. This collection enables spatial queries specifically related to Piraeus, such as identifying vessels that enter, exit, or remain within the port boundaries.

Listing 6: Example document from the piraeus_port collection (FeatureCollection with MultiPolygon)

```
{
  "_id": { "$oid": "678234a590f33466a5cb0fc5" },
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "MultiPolygon",
        "coordinates": [
          [
            [
              [23.6211246,37.9367436],
              [23.6211683,37.936639],
              [23.6211406,37.9366305],
              [23.6213119,37.9362654],
              [23.6215472,37.9363241],
              [23.6222302,37.9345956],
              [23.6225582,37.9346891],
              [23.6234921,37.93445],
```

```
              [23.62467,37.9372383],
              [23.624917,37.937821],
              [23.6257951,37.9381988],
              [23.6258464,37.9381235],
              [23.6249982,37.937755],
              [23.6247707,37.9372197],
              [23.6248508,37.9370932],
              [23.625902,37.9367617],
              [23.6267146,37.9369933],
              [23.6267836,37.9371174],
              ...
            ]
          ],
          [
            [
              [23.6214461866084,37.940555777528836],
              [23.6214472,37.9405931],
              [23.62144706749831,37.94059313526924],
              [23.6214461866084,37.940555777528836]
            ]
          ],
          [
            [
              [23.621410302057377,37.93923417577168],
              [23.621411346493115,37.93927264157564],
              [23.6214103,37.9392342],
              [23.621410302057377,37.93923417577168]
            ]
          ],
          [
            [
              [23.621254,37.936158],
              [23.6209926,37.9367085],
              [23.621000531746205,37.93672686825437],
              [23.6209925,37.9367085],
              [23.621254,37.936158]
            ]
          ]
        ]
      },
      "properties": null,
      "id": 0
    }
  ]
}
```

3.7 Collection `regions`

The `regions` collection stores data in GeoJSON FeatureCollection format. Each feature is represented using MultiPolygon geometry. This makes it possible to describe complex areas consisting of multiple polygons, including maritime zones or land areas.

Listing 7: Example document from the regions collection (FeatureCollection with MultiPolygon in projected coordinates)

```
{
  "_id": { "$oid": "6782374e90f33466a5cb0fc9" },
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "MultiPolygon",
        "coordinates": [
          [
            [
              [468376.5888746479, 4578030.497750119],
              [468365.8688747241, 4577962.997750262],
              [468363.1888747929, 4577898.997750392],
              [468368.4988748394, 4577851.997750477],
              [468380.9688748695, 4577818.497750534],
              [468440.5988749116, 4577750.997750603],
              [468721.33887507376, 4577465.497750871],
              [468807.8788751461, 4577356.497750996],
              [468905.8388752576, 4577204.497751192],
              [468959.74887535296, 4577089.497751361],
              ...
              [471636.08887775027, 4573564.997755517]
            ]
          ]
        ]
      },
      "properties": null
    }
  ]
}
```

3.8 Collection `islands`

The `islands` collection contains geospatial data for the islands in the study area. Each document is stored in GeoJSON format with Polygon geometry so that the outline of each island can be represented accurately. The fields include the island name, which enables convenient identification and filtering.

Listing 8: Example document from the islands collection (WGS84 GeoJSON Polygon)

```
{
  "_id": { "$oid": "678ff5aedf397241e190b00a" },
  "feature_id": 0,
  "geometry": {
    "type": "Polygon",
    "coordinates": [
      [
        [23.0879522, 37.8663437],
        [23.0879819, 37.8666887],
        [23.0877825, 37.8670707],
        [23.0868324, 37.867655],
        [23.0861539, 37.8682495],
        [23.0848832, 37.8688825],
        [23.084114, 37.8690178],
        [23.083711, 37.8688715],
        [23.0834872, 37.8683788],
        [23.0836716, 37.867876],
        [23.0842246, 37.8673831],
        [23.0855551, 37.8665963],
        [23.0870772, 37.8661762],
        [23.0875561, 37.8662014],
        [23.0879522, 37.8663437]
      ]
    ]
  },
  "properties": {
```

```
    "island_nam": "Plateia"
  }
}
```


3.9 Collection `spatial_coverage`

The `spatial_coverage` collection stores geospatial areas representing AIS reception coverage from specific receivers or data sources. Each document is represented as a GeoJSON FeatureCollection with Polygon or MultiPolygon geometry describing the area in which AIS signals can be collected. The collection is particularly useful for assessing data quality and completeness, because recorded observations can be associated with the corresponding geographical coverage.

Listing 9: Example document from the spatial_coverage collection (GeoJSON Polygon)


```
{
  "_id": { "$oid": "6791d829df397241e190b0a0" },
  "geometry": {
    "type": "Polygon",
    "coordinates": [
      [
        [22.99473205228611, 37.872745890640296],
        [23.107431155438775, 37.872745890640296],
        [23.107431155438775, 37.91624174007503],
        [23.105591299999997, 37.91626939999999],
        [23.103969400000004, 37.91611089999999],
        [23.102710599999998, 37.915857],
        [23.1012515, 37.9158928],
        [23.100124199999996, 37.91582929999999],
        [23.098897199999996, 37.91562429999999],
        [23.0979865, 37.915560299999996],
        [23.097383900000004, 37.91541299999998],
        [23.096704099999997, 37.9153589],
        [23.0957184, 37.91516550000001],
        [23.09505, 37.91515909999998],
        [23.094837099999996, 37.9151791],
        [23.0945316, 37.915154099999995],
        [23.0943231, 37.91523759999998],
        [23.093756399999997, 37.915291599999996],
        [23.0935343, 37.9154158],
        [23.0933599, 37.91565999999999],
        [23.093176999999997, 37.91626560000001],
        [23.0927865, 37.916970000000006],
        ...
      ]
    ]
  },
  "properties": {
    "area_id": 1,
    "population": 53228
  }
}
```


3.10 Collection `noaa_weather`

The `noaa_weather` collection stores meteorological reanalysis measurements that are point-based in both time and space, represented as GeoJSON `Feature` documents. Each document contains a `Point` geometry in WGS84 and meteorological variables inside `properties`, including temperature, wind, humidity, pressure, and precipitation. Two representations of time are preserved: `timestamp` as a string, e.g. `"YYYY-MM-DD hh:mm:ss"`, and `timestamp_` as epoch seconds for efficient time-range filtering.

Listing 10: Example document from the noaa_weather collection (GeoJSON Point)


```
{
  "_id": { "$oid": "678c3d887517bd9d370acd16" },
  "type": "Feature",
  "originalId": { "$oid": "678232bc90f33466a5cb0fc1" },
  "geometry": {
    "type": "Point",
    "coordinates": [ 23.04973, 37.86353 ]
  },
  "properties": {
    "timestamp": "2017-08-01 00:00:00",
    "timestamp_": 1501545600,
    "lon": 23.04973,
    "lat": 37.86353,
    "TMP": 299.79156,
    "TMIN": 299.17084,
    "TMAX": 299.77084,
    "PRMSL": 101655.71,
    "RH": 72.119156,
    "GUST": 5.2540326,
    "DPT": 294.34576,
    "WSPD": 4.063478305448736,
    "WDIRMAT": 102.89263145663146,
    "WDIRMET": 167.10736854336852,
    "APCP": 0.875,
    "UGRD": -0.9066626,
    "VGRD": 3.961037600000001
  }
}
```


Notes:

- *Units (indicative):* Temperatures `TMP`, `TMIN`, `TMAX`, and `DPT` are expressed in Kelvin. Pressure `PRMSL` is expressed in Pa; wind variables `WSPD`, `GUST`, `UGRD`, and `VGRD` are expressed in m/s; and `APCP` is expressed in mm/hour, depending on the product.
- *Time:* The epoch-based `timestamp_` field is preferred for range queries.

4 Indexes & Optimization

The database indexes were designed around the main use cases of the system: spatial search, temporal analysis, especially vessel-specific time-series access, metadata filtering, and data enrichment through aggregation pipelines. The main aspects of the indexing strategy are:

- **Geospatial indexes** using `2dsphere` to accelerate operations such as `$geoWithin`, `$geoIntersects`, and `$near` on collections containing geometric objects.
- **Compound indexes on (vessel_id, timestamp)** to optimize temporal queries per vessel, particularly when retrieving the “latest position” using `sort/limit`.

- **Metadata indexes** on fields such as `country`, `shiptype.Type Code`, and `locode` for efficient filtering and grouping.
- **Sparse indexes** for optional or incomplete fields, such as `name`, `mmsi`, and `feature_id`, in order to reduce index storage overhead.
- **Text indexes** for simple free-text searches over vessel, harbour, or island names.
- **Hashed indexes** for sharding with balanced workload distribution in aggregations or lookups performed by vessel.

The selection and combination of indexes aim to reduce response time for realistic operational requirements while avoiding excessive index storage overhead or a major negative impact on write performance. The following subsections present the indexes used for each collection.

4.1 `vessels`

Listing 11: Indexes on the vessels collection

```
/* 1) primary key for joins with positions */
db.vessels.createIndex({ vessel_id: 1 }, { unique: true, name: "uid_vessel_id" });

/* 2) filters by country */
db.vessels.createIndex({ country: 1 }, { name: "idx_country" });

/* 3) filters by AIS type (some docs do not contain it -> sparse) */
db.vessels.createIndex({ "shiptype.Type Code": 1 }, { sparse: true, name: "idx_shiptype_code" });

/* 4) compound filter (e.g. all Greek cargo/tug vessels) */
db.vessels.createIndex(
  { country: 1, "shiptype.Type Code": 1 },
  { name: "idx_country_shiptype" }
);

/* 5) text search */
db.vessels.createIndex({ name: "text" }, { name: "text_name" });

/* Optional for sharded lookups */
db.vessels.createIndex({ vessel_id: "hashed" }, { name: "hs_vessel_id" });
```

4.2 `ship_types`

Listing 12: Index on the ship_types collection

```
db.ship_types.createIndex({ "Type Code": 1 }, { unique: true, name: "uid_type_code" });
```

4.3 Enriching `vessels` from `ship_types`

Listing 13: Lookup/merge of vessel-type descriptions

```
db.vessels.aggregate([
  { $lookup: {
      from: "ship_types",
      localField: "shiptype.Type Code",
      foreignField: "Type Code",
      as: "t"
  }},
  { $set: {
      "shiptype.Description": {
        $ifNull: [ "$shiptype.Description", { $first: "$t.Description" } ]
      }
  }},
  { $unset: "t" },
  { $merge: { into: "vessels", on: "_id", whenMatched: "merge", whenNotMatched: "discard" } }
]);
```

4.4 `harbours`

Listing 14: Indexes on the harbours collection

```
/* Geospatial */
db.harbours.createIndex({ geometry: "2dsphere" }, { name: "geo_2dsphere" });

/* Unique locode where present */
db.harbours.createIndex({ locode: 1 }, { unique: true, sparse: true, name: "uid_locode" });

/* If names are stored under properties */
db.harbours.createIndex({ "properties.Port Name": 1 }, { name: "idx_prop_port_name" });
db.harbours.createIndex({ "properties.type": 1 }, { name: "idx_prop_type" });
```

4.5 `islands`

Listing 15: Indexes on the islands collection

```
/* Geospatial searches (near / intersects) */
db.islands.createIndex({ geometry: "2dsphere" }, { name: "geo_2dsphere" });

/* Fast filtering/searching by name */
db.islands.createIndex({ name: 1 }, { sparse: true, name: "idx_name" });

/* Optional: full-text search */
db.islands.createIndex({ name: "text" }, { name: "text_name" });

/* Stable key per feature */
db.islands.createIndex({ feature_id: 1 }, { unique: true, sparse: true, name: "uid_feature_id" });
```

4.6 `spatial_coverage`

Listing 16: Index on the spatial_coverage collection

```
db.spatial_coverage.createIndex({ geometry: "2dsphere" }, { name: "gix_spatial_coverage" });
```

4.7 territorial_waters

Listing 17: Index on the territorial_waters collection

```
/* If the schema is a FeatureCollection with features[].
     geometry */
db.territorial_waters.createIndex({ "features.geometry": "2
     dsphere" }, { name: "geo_2dsphere" });
```

4.8 ais_synopses

Listing 18: Indexes on the ais_synopses collection

```
/* Per vessel and time (range queries) */
db.ais_synopses.createIndex({ vessel_id: 1, t: 1 }, { name: "
     vessel_ts" });

/* For bbox + time (useful for lon/lat + t range scans) */
db.ais_synopses.createIndex({ lon: 1, lat: 1, t: 1 }, { name:
      "bbox_ts" });
```

4.9 regions

Listing 19: Indexes on the regions collection

```
db.regions.createIndex({ geometry: "2dsphere" }, { name: "
     gix_regions" });
db.regions.createIndex({ name: 1 }, { name: "idx_name" });
```

4.10 ais_dynamic

Listing 20: Indexes on the ais_dynamic collection

```
/* Geospatial */
db.ais_dynamic.createIndex({ geometry: "2dsphere" }, { name:
     "gix_geom" });

/* Latest positions per vessel (sort/limit) */
db.ais_dynamic.createIndex({ vessel_id: 1, timestamp: -1 }, {
      name: "idx_vid_time_desc" });

/* Time windows */
db.ais_dynamic.createIndex({ timestamp: -1 }, { name: "
     idx_time_desc" });

/* Optional: prevent duplicate records */
db.ais_dynamic.createIndex({ vessel_id: 1, timestamp: 1 }, {
     unique: true, name: "uid_vid_time" });
```

Performance comment. The `2dsphere` indexes substantially accelerate `$geoWithin`/`$near` and spatial filters involving time when combined with a `t/timestamp` field. Compound `(vessel_id, timestamp)` indexes improve time-series scans per vessel, especially when `sort/limit` is used to retrieve the latest observations. In sharded clusters, a `hashed` index on `vessel_id` supports more uniform workload distribution for per-vessel lookups and aggregations.

5 Sharding

5.1 Cluster Topology

An **Atlas sharded cluster** was used (MongoDB 8.0, M30 tier) with:

- **Routers (mongos):** 1, represented by the Atlas SRV connection string.
- **Shards:** 2 shards, each implemented as a 3-member replica set: `atlas-v0ocfg-shard-0` and `config`[1].
- **Config servers:** 3× `configsvr` nodes (replica set `cfgRS`).
- **Chunk size:** Atlas default (128 MB).

5.2 Shard-Key Selection for zenodoDB.ais_dynamic

Two strategies were considered:

(1) **Hashed** `vessel_id` — provides a good uniform distribution of writes and helps prevent hot shards when a small number of `vessel_id` values generate a disproportionately high update rate.

(2) **Compound range key** `{vessel_id:1, timestamp:1}` — optimizes range queries per vessel.

Selected strategy: Hashed `vessel_id`. The main criterion was that AIS ingestion is write-heavy on a per-vessel basis and an even distribution is desirable.

Supporting indexes are maintained for the query workload:

```
db.ais_dynamic.createIndex({ geometry: "2dsphere" });
db.ais_dynamic.createIndex({ vessel_id: 1, timestamp: -1 });
db.ais_dynamic.createIndex({ timestamp: -1 });
```

5.3 Procedure (mongosh)

```
use admin;
sh.enableSharding("zenodoDB");

// 1) Create an index aligned with the hashed shard key
use zenodoDB;
db.ais_dynamic.createIndex({ vessel_id: "hashed" },{ name:"
     vessel_id_hashed" });

// 2) Declare the collection as sharded
sh.shardCollection("zenodoDB.ais_dynamic", { vessel_id: "
     hashed" });

// 3) Verification
sh.status();                  // quick overview
db.ais_dynamic.getIndexes();  // verify index
```

5.4 Balancing & Chunk Initialization

Initially, the collection had one chunk and all data were located on a single shard. The balancer was enabled and chunks were also moved manually to accelerate redistribution:

```
sh.setBalancerState(true);

// Example chunk transfers
```

[1]The second shard appears with the id `config` in Atlas `config.chunks`.

```
const NS = "zenodoDB.ais_dynamic";
const cfg  = db.getSiblingDB("config");
const coll = cfg.collections.findOne({ _id: NS },{ uuid:1 });
cfg.chunks.find({ uuid: coll.uuid }).limit(20).forEach(ch => {
  sh.moveChunk(NS, ch.min, "config");   // move toward second shard
});
```

5.5 Reported Measurements

Chunk distribution per shard.

```
// Chunks per shard for the namespace
const NS = "zenodoDB.ais_dynamic";
const cfg  = db.getSiblingDB("config");
const coll = cfg.collections.findOne({ _id: NS },{ uuid:1 });

cfg.chunks.aggregate([
  { $match: { uuid: coll.uuid } },
  { $group: { _id: "$shard", chunks: { $sum: 1 } } },
  { $sort: { chunks: -1 } }
]).toArray();
```

Final state after the moves:

{ `config` : **36** chunks, `atlas-v0ocfg-shard-0` : **30** chunks}

Deviation from the mean (= 33 chunks): `config` +9.1%, `atlas-v0ocfg-shard-0` −9.1%.

Data size & document count per shard.

```
const s = db.getSiblingDB("zenodoDB").ais_dynamic.stats({ scale: 1024*1024 });
print("TOTAL sizeMB:", s.size.toFixed(2), " count:", s.count);
Object.entries(s.shards).forEach(([sh,v]) => {
  print(`${sh}  sizeMB=${v.size.toFixed(2)}  count=${v.count}  avgObjSizeB=${Math.round(v.avgObjSize)}`);
});
```

Measurement snapshot after rebalancing:

Shard	Size (MB)	#Docs	Avg Obj (B)
`atlas-v0ocfg-shard-0`	797.55	2,475,343	337
`config`	910.53	2,834,913	336
Total	**1708.08**	**5,310,256**	

Chunk migrations (24h).

```
db.getSiblingDB("config").changelog.aggregate([
  { $match: { what: "moveChunk.commit",
              "details.collectionUUID": coll.uuid,
              time: { $gte: new Date(Date.now() - 24*60*60*1000) } } },
  { $group: { _id: { from:"$from", to:"$to" }, moves: { $sum: 1 } } },
  { $sort: { moves: -1 } }
]).toArray();
```

After the manual moves, `moveChunk.commit` entries were observed and the balancer remained `ON`.

5.6 Correctness Checks & Practices

- **Index aligned with shard key:** `vessel_id_hashed` existed before `shardCollection`.
- **Avoiding jumbo chunks:** chunk movements/splits were used until a total of 66 chunks was reached.
- **Other collections:** these remained *unsharded* because of their comparatively small size, e.g. `islands` 1.67 MB, `vessels` 1.15 MB, and `ais_synopses` 179 MB. If necessary, `ais_synopses` could be sharded using `{day:1}` or `{timestamp:1}`.

5.7 Results

After sharding `ais_dynamic` using `vessel_id: "hashed"` and performing the initial balancing:

- **Chunk distribution:** 36 vs 30, corresponding to a deviation of ±9.1% from the mean.
- **Data distribution:** approximately 910 MB / 798 MB, i.e. roughly 53% / 47% of the records.
- **Collection total:** 5,310,256 records and 1.67 GB logical size.
- **Queries:** Equality predicates on `vessel_id` and time-window queries are supported by the compound `vessel_id`/`timestamp` index and the separate descending `timestamp` index. The hashed shard key helps prevent hotspots for "heavy" `vessel_id` values.

6 Query Implementation

This section presents representative queries over the AIS data. For each query, the corresponding index is identified in order to document the intended execution efficiency.

6.1 Relational-Style Queries

(a) Vessels registered in Greece whose name matches a term. The query uses the `text` index on `name` and the index on `country`.

Listing 21: Vessels registered in Greece whose name matches a term

```
db.vessels.find(
  { country: "Greece", $text: { $search: "AGIOS" } },
  { vessel_id: 1, name: 1, _id: 0 }
);
```

(b) Join with `ship_types` *and description filtering.* The query joins `vessels` with `ship_types` in order to filter passenger vessels. It uses indexes on `country` and `shiptype.Type Code` in `vessels`, as well as the `Type Code` index in `ship_types`.

Listing 22: Join between vessels and ship_types with exact/regex description filter

```
/* Exact description */
db.vessels.aggregate([
  { $match: { country: "Greece" } },
  { $lookup: {
      from: "ship_types",
      localField: "shiptype.Type Code",
      foreignField: "Type Code",
      as: "CodeInfo"
```

```
  }},
  { $unwind: "$CodeInfo" },
  { $match: { "CodeInfo.Description": "Passenger, all ships of this type" } },
  { $project: { _id: 0, vessel_id: 1, country: 1, shiptype: "$CodeInfo.Description" } }
]);

/* Alternatively, regex (case-insensitive) */
db.vessels.aggregate([
  { $match: { country: "Greece" } },
  { $lookup: {
      from: "ship_types",
      localField: "shiptype.Type Code",
      foreignField: "Type Code",
      as: "CodeInfo"
  }},
  { $unwind: "$CodeInfo" },
  { $match: { "CodeInfo.Description": { $regex: "Passenger", $options: "i" } } },
  { $project: { _id: 0, vessel_id: 1, country: 1, shiptype: "$CodeInfo.Description" } }
]);
```

6.2 Results (Relational-Style Queries)

A small sample of Q1-related query results is shown below.

Listing 23: Join between vessels and shiptypes – indicative results (mongosh format)

```
{
  _id: ObjectId('679f60f6f686b3331979e837'),
  vessel_id:
  '3daf9056a28b90e8ac1910357214cf713b575cda9ee118eb8f89faed1e37c1d4',
  country: 'Greece',
  shiptype: 60,
  CodeInfo: {
    _id: ObjectId('67687ec46762f23d2f7941aa'),
    'Type Code': 60,
    Description: 'Passenger, all ships of this type'
  }
}

{
  _id: ObjectId('679f60f6f686b3331979e838'),
  vessel_id:
  '9daf29c7324a1882b4ebdb556f406dfd2930cd6dcf84bbe949fe60fc9c45fd3e',
  country: 'Greece',
  shiptype: 60,
  CodeInfo: {
    _id: ObjectId('67687ec46762f23d2f7941aa'),
    'Type Code': 60,
    Description: 'Passenger, all ships of this type'
  }
}

{
  _id: ObjectId('679f60f6f686b3331979e84c'),
  vessel_id:
  '0a9674dc0bcf0d25c4809047b80ddea2b8db62a4537548cf29f17aeb8b1dc36d',
  country: 'Greece',
  shiptype: 69,
  CodeInfo: {
    _id: ObjectId('67687ec46762f23d2f7941b3'),
    'Type Code': 69,
    Description: 'Passenger, No additional information'
  }
}
```

6.3 Spatial Queries

(a) Vessels within a circle of radius R=5 km centered at (23.64, 37.94). The query uses the `2dsphere` index on `ais_dynamic.geometry`.

Listing 24: Vessels within a 5 km circle centered at (23.64, 37.94)

```
db.ais_dynamic.find({
  geometry: {
    $geoWithin: {
      $centerSphere: [[23.64, 37.94], 5 / 6378.1] // 5 km in Earth-radius radians
    }
  }
});
```

6.4 Results (Spatial Query (a): Vessels Within a Circle)

A small sample of Q2 results is shown below.

Listing 25: Repeated ais_dynamic records – example duplicates with the same time and position

```
{
  _id: ObjectId('676feed31ce3cbb7e4cee9dc'),
  vessel_id:
  '6e414ea63d244165ab6b4e74ee96b384043d0fd3f99dbfc40a400eedf0ab81fc',
  heading: 186,
  speed: 14.7,
  course: 186.6,
  timestamp: ISODate('2017-05-12T10:44:23Z'),
  geometry: {
    type: 'Point',
    coordinates: [23.622166666666693, 37.900999999999996]
  }
}

{
  _id: ObjectId('676ff0531ce3cbb7e4e56db0'),
  vessel_id:
  '6e414ea63d244165ab6b4e74ee96b384043d0fd3f99dbfc40a400eedf0ab81fc',
  heading: 186,
  speed: 14.7,
  course: 186.6,
  timestamp: ISODate('2017-05-12T10:44:23Z'),
  geometry: {
    type: 'Point',
    coordinates: [23.622166666666693, 37.900999999999996]
  }
}
```

(b) Richer result: join with `vessels`. The aggregation also returns vessel metadata in `vesselInfo`.

Listing 26: Vessels within 5 km enriched with vessels metadata

```
db.ais_dynamic.aggregate([
  { $match: {
      geometry: { $geoWithin: { $centerSphere: [[23.64, 37.94], 5/6378.1] } }
  }},
  { $lookup: {
```

```
      from: "vessels",
      localField: "vessel_id",
      foreignField: "vessel_id",
      as: "vesselInfo"
  }},
  { $unwind: "$vesselInfo" }
]);
```

6.5 Results (Spatial Query (b): Enriched Vessels Within a Circle)

A small sample of the enriched Q2 result is shown below.

Listing 27: Embedding vessel information (vesselInfo) in ais_dynamic – lookup result

```
{
  _id: ObjectId('676fed3f1ce3cbb7e4b80001'),
  vessel_id: '6
      e414ea63d244165ab6b4e74ee96b384043d0fd3f99dbfc40a400eedf0ab81fc'
      ,
  heading: 186,
  speed: 14.7,
  course: 186.6,
  timestamp: ISODate('2017-05-12T10:44:23Z'),
  geometry: {
    type: 'Point',
    coordinates: [23.622166666666693, 37.900999999999996]
  },
  vesselInfo: {
    _id: ObjectId('679f60f8f868b3331979f983'),
    vessel_id: '6
      e414ea63d244165ab6b4e74ee96b384043d0fd3f99dbfc40a400eedf0ab81fc'
      ,
    country: 'Norway',
    shiptype: 70
  }
}
```

Listing 28: Three nearest vessels to a point *k* nearest vessels (kNN)..

```
db.ais_dynamic.find({
  geometry: {
    $near: {
      $geometry: { type: "Point", coordinates: [23.65, 37.94]
            },
      $maxDistance: 5000
    }
  }
}).limit(3);
```

6.6 Results of Spatial Query: *k* Nearest Vessels (kNN)

A small sample of Q3 results is shown below.

Listing 29: Regular ais_dynamic records without heading/denormalized fields

```
{
  _id: ObjectId('676fee541ce3cbb7e4c67a36'),
  vessel_id: '
      a6137c45b3d5fce9f291e4f22ffa4a81fff2cebe6606c3e2342dc14ea15bbc03
      ',
  speed: 0,
  course: 348.5,
  timestamp: ISODate('2017-05-20T16:43:31Z'),
  geometry: {
    type: 'Point',
    coordinates: [23.6493616666667, 37.938473333333]
  }
}

{
  _id: ObjectId('676fee691ce3cbb7e4c814fb'),
  vessel_id: '
      a6137c45b3d5fce9f291e4f22ffa4a81fff2cebe6606c3e2342dc14ea15bbc03
      ',
  speed: 0,
  course: 305,
  timestamp: ISODate('2017-05-21T05:14:21Z'),
  geometry: {
    type: 'Point',
    coordinates: [23.6493283333333, 37.9384766666667]
  }
}
```

Listing 30: Three nearest vessels with distance and vesselInfo (d) *k* nearest vessels with distance & enrichment (aggregation).

```
db.ais_dynamic.aggregate([
  { $geoNear: {
      near: { type: "Point", coordinates: [23.65, 37.94] },
      distanceField: "distance",
      maxDistance: 5000,
      spherical: true,
      key: "geometry"
  }},
  { $lookup: {
      from: "vessels",
      localField: "vessel_id",
      foreignField: "vessel_id",
      as: "vesselInfo"
  }},
  { $unwind: "$vesselInfo" },
  { $limit: 3 }
]);
```

6.7 Spatio-Temporal Queries

(a) Vessels within 2 km and a specified time window. The query uses the `2dsphere` index on `geometry` and an index on `timestamp`.

Listing 31: Vessels within 2 km and a time window

```
db.ais_dynamic.find({
  timestamp: {
    $gte: ISODate("2017-05-09T14:00:00Z"),
    $lte: ISODate("2017-05-09T15:00:00Z")
  },
  geometry: {
    $near: {
      $geometry: { type: "Point", coordinates: [23.64, 37.94]
            },
      $maxDistance: 2000
    }
  }
});
```

6.8 Spatio-Temporal Query Results

A small sample of Q4 results is shown below.

Listing 32: Nearby-vessel search results – enrichment with distance and candidate list

```
{
  _id: ObjectId('676fecad1ce3cbb7e4ac49c3'),
  vessel_id: '90
      f4a51882f8dedd4f48edf866be58b42f8fce6fb87ae96590d247889dea70d4',
  speed: 0.1,
  course: 185.5,
  timestamp: ISODate('2017-05-09T15:06:41Z'),
  geometry: {
    type: 'Point',
    coordinates: [23.648021666666693, 37.938188333333294]
  },
  distance: 266.1542688650586,
  closeShips: [
    {
      _id: ObjectId('676fecad1ce3cbb7e4ac48cc'),
      vessel_id: '
      ce1b88a7983b03e73811ccc8ea111f6879a89ae04eca20ee8f6a8c1c080917b5
      ',
      speed: 0,
      course: 185.4,
      timestamp: ISODate('2017-05-09T15:05:26Z'),
      geometry: {
        type: 'Point',
        coordinates: [23.63614, 37.9399033333333]
      }
    }
  ]
}
```

(b) Approach to an island (e.g. Aegina) within a radius of 2 km. The query uses a `2dsphere` index on `regions.geometry`.

Listing 33: Approach to an island (Aegina) within 2 km

```
const island = db.regions.findOne({ name: "Aegina" });
db.ais_dynamic.find({
  geometry: {
    $near: { $geometry: island.geometry, $maxDistance: 2000 }
  }
});
```

(c) Candidate vessel pairs that approached within distance X during time interval T. The `$geoNear` stage identifies nearby observations around a point, while `$lookup` retrieves other observations in the same time window, producing candidate vessel pairs. Note that `$geoNear` can only appear as the **first** stage of a `pipeline`.

Listing 34: Candidate vessel pairs close in space and time

```
db.ais_dynamic.aggregate([
  { $geoNear: {
      near: { type: "Point", coordinates: [23.65, 37.94] },
      distanceField: "distance",
      maxDistance: 500,
      spherical: true,
      key: "geometry"
  }},
  { $match: {
      timestamp: {
        $gte: ISODate("2017-05-09T15:05:26Z"),
        $lte: ISODate("2017-05-09T15:07:27Z")
      }
  }},
  { $lookup: {
      from: "ais_dynamic",
      let: { ship1: "$vessel_id" },
      pipeline: [
        { $match: {
            timestamp: {
              $gte: ISODate("2017-05-09T15:05:26Z"),
              $lte: ISODate("2017-05-09T15:07:27Z")
            },
            $expr: { $ne: [ "$vessel_id", "$$ship1" ] }
        }}
      ],
      as: "closeShips"
  }}
], { allowDiskUse: true });
```

6.9 Execution and Index Verification

To document execution efficiency, `explain` is used.

```
db.vessels.find({ country: "Greece" }).explain("
    executionStats")
db.ais_dynamic.find({
  geometry: { $near: { $geometry: { type:"Point", coordinates
      :[23.64,37.94] } } }
}).explain("executionStats")
```

6.10 Performance Comments

- Spatial queries (`$geoWithin`, `$near`, `$geoNear`) are accelerated by the `2dsphere` index on `ais_dynamic.geometry`.
- Spatio-temporal filters benefit from the compound `(vessel_id, timestamp)` index for time-series scans, e.g. "latest positions".
- Lookups involving `ship_types` scale effectively because indexes are available on `ship_types("Type Code")` and `vessels("shiptype.Type Code")`.

7 Experimental Evaluation

7.1 Methodology

Environment. MongoDB **8.0.13** (`mongod --version`), storage engine: **WiredTiger**. Atlas (Linux, Amazon Linux 2), `aarch64`, **2 vCPU**, **7.5GB RAM** (`db.hostInfo()`). The sharded cluster consisted of: 1 shard `atlas-v0ocfg-shard-0`, while the `config` replica set also hosted chunks of the collection. The `zenodoDB.ais_dynamic` collection was *sharded* using the key `{vessel_id: "hashed"}` and approximately 15 chunks (8 on `atlas-v0ocfg-shard-0`, 7 on `config`).

Measurements. Latency was measured using wall-clock benchmarking with the helper function `bench(fn, rounds=30)` and a sample of 30 executions. For scan/return cost, `explain("executionStats")` was used. The helper `xr(cur)` returns `totalDocsExamined` and `nReturned`.

7.2 Results

Table 2: Latency (ms) and scanned/returned documents for representative queries

Query	Index	p50	p95	Scanned	Returned
Q1 (text + country)	`text_name` + `idx_country`	45	103	1234	987
Q2 (geoWithin 5 km)	`gix_geom`	80	1267	2783	2000
Q3 (near $k = 10$)	`gix_geom`	1734	2055	319519	20
Q4 (geo+time)	`gix_geom` + `idx_time_desc`	2618	3098	1094104	2000

p50/p95: latency percentiles (ms). Scanned = documents examined, Returned = documents in the cursor (limit=2000).

Measurement methodology. Each query was executed 30 times against the cache with `limit=2000`. **p50/p95** denote the median and 95th percentile of client-side wall-clock latency in milliseconds. **Returned** is the number of documents returned by the cursor, capped at 2000 by the limit. **Scanned** corresponds to `executionStats.totalDocsExamined` from `.explain("executionStats")`. In *Atlas Search*, this metric is not exposed and is represented as "—". An efficiency indicator can be defined as $\eta = \text{Scanned}/\text{Returned}$, where values close to 1 are desirable.

Comments & interpretation by query. **Q1 (text + country).** The low p50/p95 values are attributed to the inverted text index: the term is located directly and the equality predicate on country rapidly restricts the result set. When the term does not match exactly, e.g. "AGIOS" versus "Agios/Ágios", *Returned* may be 0. For tolerant matching and spelling variants, `$search` with *fuzzy/autocomplete* or an appropriate analyzer is preferable. (*Note:* `$search` does not expose the *Scanned* metric.)

Q2 ($geoWithin 5 km). The `2dsphere` index (`gix_geom`) restricts scanning to cells intersecting the query circle. Therefore, *Scanned* is relatively close to *Returned* and latency remains comparatively low. In dense locations such as harbour areas, *Returned* quickly reaches the `limit`, while *Scanned* grows accordingly without the much larger amplification observed in the less selective queries.

Q3 ($near, k=10). The `$near` operation expands its search until k points are found. Without `$maxDistance`, many cells/documents may be examined, especially in sparse areas. This is reflected in the much larger *Scanned* value and the difference between p50 and p95, which represents tail latency. *Potential improvement:* define `$maxDistance`, e.g. 2–5 km, or pre-filter using `$geoWithin` before applying `$near` to the reduced subset.

Q4 (geo + time). When the temporal predicate is applied after the spatial condition *without* an appropriate compound index, a large intermediate result set may be examined before temporal filtering occurs, resulting in a high *Scanned* value and increased p95 latency. *Potential improvements:* (i) use `$geoNear` as the first stage with a small `maxDistance`, (ii) use a compound index such as `{ geom:"2dsphere", ts:-1 }` to support index-level temporal filtering/sorting, and (iii) use time bucketing, e.g. hourly, or a `timeSeries` collection.

Comparative conclusions. Q1–Q2 exhibit a relatively low efficiency ratio η due to targeted indexes. Q3 without `maxDistance` and Q4 without an effective compound index exhibit substantially higher numbers of scanned documents per result and higher tail latency. The main lesson is to **align** filters with appropriate compound indexes, particularly spatial+time indexes, to **restrict the search range** in kNN scenarios, and to use **analyzers** for text search when spelling or transliteration variants must be supported.

Threats to validity. (a) The measurements are affected by data density, geographical location, and cache state; (b) `limit` can hide the actual number of matching results in dense areas; (c) for Atlas Search, the *Scanned* metric is not available and should therefore be reported explicitly as "—".

8 Conclusions & Future Work

Conclusions. Modeling the data with `GeoJSON` and targeted indexing enabled efficient query execution in spatial and spatio-temporal scenarios. The main findings are:

- **`2dsphere`** indexes on `geometry` are critical: `$geoWithin` and `$near`/`$geoNear` can use index-based execution plans where possible, substantially reducing latency.
- Compound **time-series** indexes on `(vessel_id, timestamp)` significantly improve per-vessel range scans and support "latest position" queries using `sort/limit` without expensive full scans.
- **Sharding** using a hashed `vessel_id` distributes writes more uniformly and helps prevent hotspots when a small number of vessels generate a disproportionately large number of signals. Supporting indexes on `geometry` and `timestamp` remain necessary for the query workload.
- **Text queries.**
  A simple `text` index is sufficient for exact terms when `default_language` is set to `"none"`. For variants, diacritics, or transliterations, `regex` with an appropriate `collation` is more practical.
- In kNN scenarios, `$near`, or `$geoNear` inside a pipeline, provides faster and accurate results when combined with `limit` and a small `maxDistance`; without these constraints, scan cost increases.

Limitations.

- Some geospatial collections, such as `regions`, provide *projected* coordinates; reprojection to WGS84 is required for full use of `2dsphere`.
- Text fields are incomplete in a number of `vessels` documents, which limits the effectiveness of `$text`.

Future work.

- **Bucketing/Compression** in `ais_dynamic`, e.g. hourly buckets or columnar `timeseries` collections, to reduce storage and accelerate range scans.
- **Re-evaluation of the shard key.** Investigate `{vessel_id:1, timestamp:1}` as a *compound range* key for workloads dominated by time-window queries.
- **Atlas Search**, where available, for richer full-text search using analyzers, synonyms, and fuzzy matching on `vessels.name`/`callsign`.

- **Precomputed spatial datasets** or materialized views, such as “latest position per vessel” or “entry/exit from zones”, to serve frequently used dashboards more efficiently.
- **Full geometry harmonization** by maintaining WGS84 geometry in `geometry_wgs84` for every collection so that transformations are not required at query time.
- **Filtered indexing** using `partial indexes` for scenarios focusing on particular time intervals or subsets of vessels.
- **Monitoring/Auto-tuning** using the Profiler, `explain`, and `serverStatus`, together with periodic index review based on actual query patterns.

References

[1] Pence, Harry E. ”What is big data and why is it important?.” Journal of Educational Technology Systems 43.2 (2014): 159-171.

[2] Svanberg, Martin, et al. ”AIS in maritime research.” Marine Policy 106 (2019): 103520.

[3] Tritsarolis, Andreas, Yannis Kontoulis, and Yannis Theodoridis. ”The Piraeus AIS dataset for large-scale maritime data analytics.” Data in Brief 40 (2022): 107782.

[4] Maritime Activity—AIS Dataset for the Saronic Gulf, Greece. Zenodo, 2019. https://doi.org/10.5281/zenodo.6323416.

[5] Pokorny, Jaroslav. ”NoSQL databases: a step to database scalability in web environment.” Proc. iiWAS. 2011.

[6] Parker, Zachary, Scott Poe, and Susan V. Vrbsky. ”Comparing NoSQL MongoDB to an SQL DB.” Proc. ACM Southeast. 2013.